

WEEK 2 DOCUMENTATION FINDINGS

Intern: Muhammad Saim Ali Khan

ID: DHC-4044

Role: Cyber Security Analyst Intern

1. FIXING VULNERABILITIES

To address the injection risks identified in Week 1, I implemented strict input validation and secure credential storage.

A. Input Sanitization and Validation

Using the `validator` library, I ensured that all user inputs are checked for empty strings and malicious characters before reaching the database.

JAVASCRIPT:

```
const validator = require('validator');
const bcrypt = require('bcrypt');

app.post('/register', async function(req, res) {
  const { username, email, password } = req.body;

  if (!username || !email || !password) {
    return res.status(400).send("All fields are required.");
  }

  if (!validator.isEmail(email)) {
```

```

    return res.status(400).send("Invalid email format.");
  }

  const hashedPassword = await bcrypt.hash(password, 10);

  pool.query('INSERT INTO users (username, email, password) VALUES
  (?, ?, ?)',
  [username, email, hashedPassword], (err, result) => {
    if (err) return res.status(500).send("Server Error");
    res.status(201).send("User Registered Successfully");
  });
});

```

2. ENHANCING AUTHENTICATION

I replaced simple session-based logins with **JSON Web Tokens (JWT)** and implemented middleware to protect sensitive CRUD operations.

A. JWT Middleware (Middleware/authenticateToken.js)

This script verifies the token's validity before allowing access to a route.

JAVASCRIPT:

```

const jwt = require('jsonwebtoken');

function authenticateToken(req, res, next) {

```

```
const authHeader = req.headers['authorization'];
const token = authHeader && authHeader.split(' ')[1];

if (!token) return res.status(401).send("Access Denied: No Token
Provided");

jwt.verify(token, 'your-secure-secret-key', (err, user) => {
  if (err) return res.status(403).send("Invalid or Expired Token");
  req.user = user;
  next();
});
}
```

module.exports = authenticateToken;

B. Protecting CRUD Operations

The following implementation shows the `/list` and `/delete` routes secured by the authentication middleware.

JAVASCRIPT:

```
app.get('/list', authenticateToken, (req, res) => {
  pool.query('SELECT id, username, email FROM users', (err, result) => {
    if (err) return res.status(400).send("Error fetching list");
```

```
    res.status(200).send(result);
  });
});

app.delete('/delete/:id', authenticateToken, (req, res) => {
  const userId = req.params.id;
  pool.query('DELETE FROM users WHERE id = ?', [userId], (err, result)
=> {
    if (err) return res.status(400).send("User not found");
    res.status(200).send(`User ${userId} removed successfully`);
  });
});
```

3. SECURING DATA TRANSMISSION

I integrated the `helmet` middleware to automatically set security headers that mitigate common web-based attacks.

```
const helmet = require('helmet');
const app = express();

app.use(helmet());
```